

# SQL Statement Fundamentals

This note will focus on SQL syntax, which is really important as they can be applied to any major type of SQL databases (MySQL, Oracle, etc).

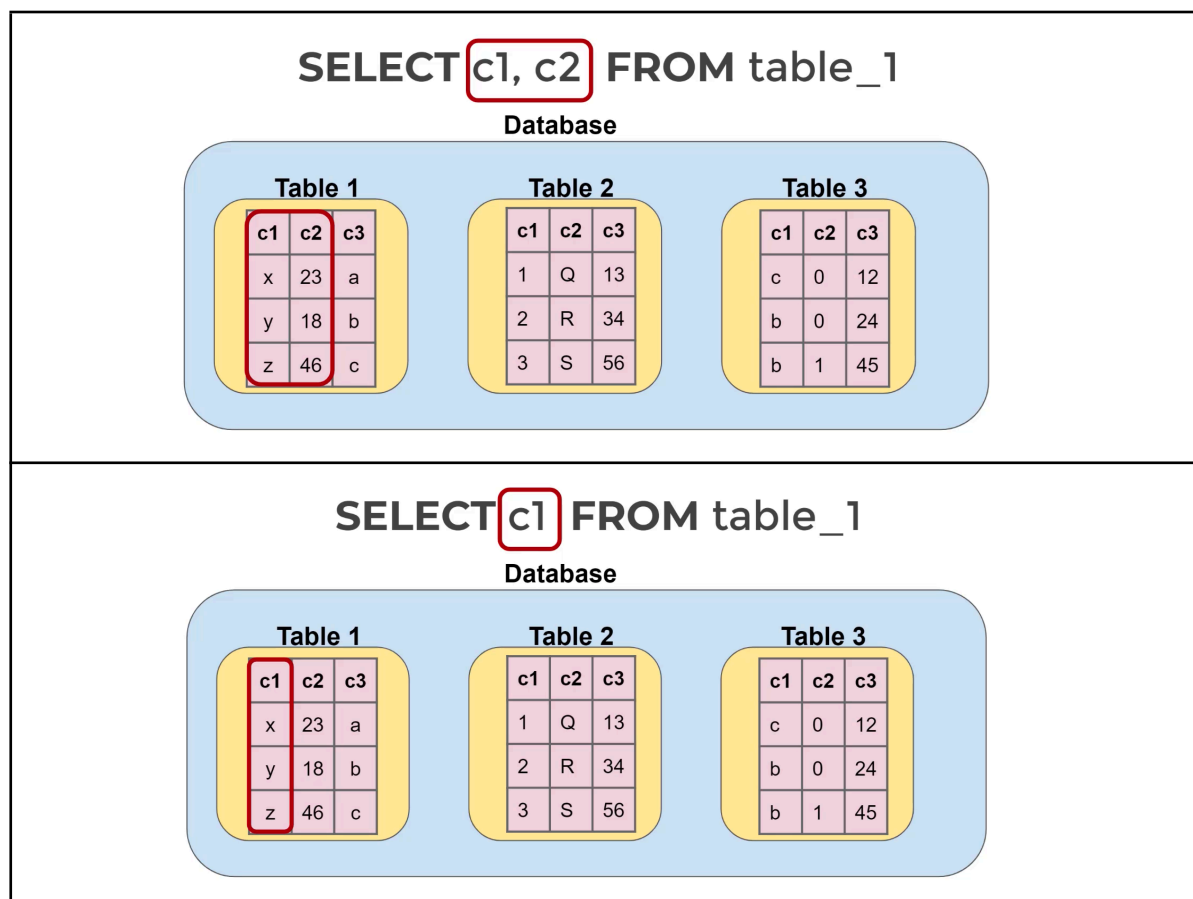
## SELECT

**SELECT** is the most common statement used and it allows us to retrieve information from a table. Also we can combine the **SELECT** statement with other statements to perform more complex queries.

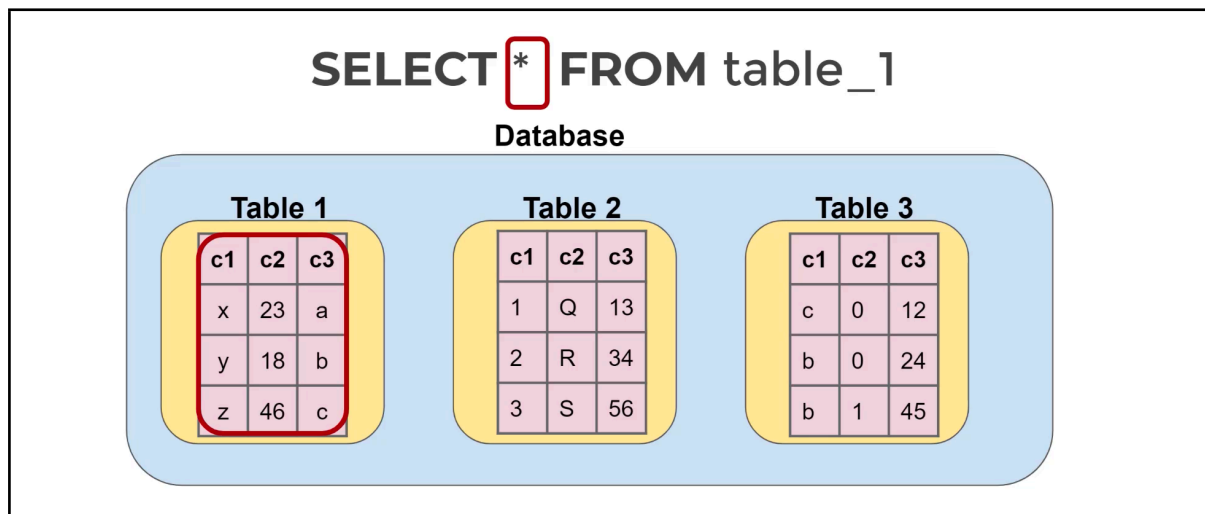
Example syntax:

```
SELECT column_name FROM table_name
```

Notice that in general, we capitalize SQL keywords but it's not a necessity. This capitalization helps us to read the query more easily by differentiating: fundamental SQL syntax statements and the names of columns and tables.



You can use the \* symbol to retrieve all the columns from the specified table.



In general it is not good practice to use an asterisk (\*) in the **SELECT** statement if we don't really need all columns because it can automatically query everything, which increases traffic between the database server and the application, which can slow down the retrieval of results.

## SELECT DISTINCT

Sometimes a table can contain a column that has duplicate values, and you may find yourself in a situation where you only want to list the unique / distinct values. The **DISTINCT** keyword can be used to return only the distinct values in a column.

Example syntaxes:

```
SELECT DISTINCT(column) FROM table;
```

```
SELECT DISTINCT column FROM table;
```

- It will work with or without parenthesis.
- Later on when we learn about adding more calls such as **COUNT** and **DISTINCT** together, the parenthesis will be necessary

Imagine a table of people who were surveyed to choose a color:

| Name   | Choice |
|--------|--------|
| Zach   | Green  |
| David  | Green  |
| Claire | Yellow |
| David  | Red    |

Notice that there are two instances of David inside this name column.

**SELECT DISTINCT name FROM color\_table**

| Name   |
|--------|
| Zach   |
| David  |
| Claire |

- Given the above example, we don't really know if the person with the name “David” was a duplicate entry, or two different people with the same first name.
- Calling **DISTINCT** here answered the question: *What are the unique first names that are in the table ?*

It makes more sense to ask “How many types of unique color choices were there?”

| Name   | Choice |
|--------|--------|
| Zach   | Green  |
| David  | Green  |
| Claire | Yellow |
| David  | Red    |

**SELECT DISTINCT choice**  
**FROM color\_table**

| Choice |
|--------|
| Green  |
| Yellow |
| Red    |

## COUNT

---

The **COUNT** function returns the number of input rows that match a specific condition of a query. We can apply **COUNT** on a specific column or just pass **COUNT(\*)**, which will return the same result.

**SELECT COUNT(name) FROM table;**

| Name   | Choice |
|--------|--------|
| Zach   | Green  |
| David  | Green  |
| Claire | Yellow |
| David  | Red    |

**SELECT COUNT(name) FROM table;**

| Count |
|-------|
| 4     |

Notice that **COUNT** needs a parenthesis as it is a function acting on something.

- This simply returns the number of rows in the table.
- It should return the same result, regardless of the column because in a table each column always has the same number of rows.

**SELECT COUNT(name) FROM table;**  
**SELECT COUNT(choice) FROM table;**  
**SELECT COUNT(\*) FROM table;**  
All return the same thing, since the original table had 4 rows.

| Count |
|-------|
| 4     |

- **COUNT** by itself simply returns back a count of the number of rows in a table.
- **COUNT** is much more useful when combined with other commands, such as **DISTINCT**.

Imagine we wanted to know: *How many unique names are there in the table?*

| Name   | Choice |
|--------|--------|
| Zach   | Green  |
| David  | Green  |
| Claire | Yellow |
| David  | Red    |

**SELECT COUNT(DISTINCT name)**  
**FROM table;**

| Count |
|-------|
| 3     |

- Firstly, **DISTINCT** finds out the unique names from the name column.
- Finally, the **COUNT** is going to count the remaining rows.
- **COUNT(DISTINCT name)** : Calls **COUNT** on the result of **DISTINCT** name.
- You can also use the below query as it gives the same result:

```
SELECT COUNT(DISTINCT(name)) FROM table;
```

## SELECT & WHERE

---

**SELECT** and **WHERE** are the most fundamental SQL statements and you will find yourself using them often. The **WHERE** statement allows us to specify conditions on columns for the rows to be returned.

Basic Syntax Example:

```
SELECT column1, column2 FROM table WHERE conditions;
```

The **WHERE** clause appears immediately after the **FROM** clause of the **SELECT** statement. The conditions are used to filter the rows returned from the **SELECT** statement.

Condition Operators:

### Comparison Operators

- Compare a column value to something.
  - Is the price *greater than* \$3.00?
  - Is the pet's name *equal to* **"Sam"**?

## Comparison Operators

| Operator | Description              |
|----------|--------------------------|
| =        | Equal                    |
| >        | Greater than             |
| <        | Less Than                |
| >=       | Greater than or equal to |
| <=       | Less than or equal to    |
| <> or != | Not equal to             |

## Logical Operators

- Allow us to combine multiple comparison operators
  - **AND**
  - **OR**
  - **NOT**

Simple Syntax Example:

**SELECT** name,choice **FROM** table  
Now let's get only the people named David

| Name   | Choice |
|--------|--------|
| Zach   | Green  |
| David  | Green  |
| Claire | Yellow |
| David  | Red    |



**SELECT** name,choice **FROM** table  
**WHERE** name = 'David'

| Name  | Choice |
|-------|--------|
| David | Green  |
| David | Red    |

**SELECT** name **FROM** table  
**WHERE** name = 'David'

| Name  |
|-------|
| David |
| David |

**SELECT** name,choice **FROM** table  
**WHERE** name = 'David' **AND** choice= 'Red'

| Name  | Choice |
|-------|--------|
| David | Red    |

## ORDER BY

---

Sometimes postgresql returns the same request query results in different order. You can control the order of the SQL query results using the **ORDER BY** keyword, which sorts the rows based on column values, in either ascending or descending.

Basic Syntax Example:

```
SELECT column_1, column_2 FROM table ORDER BY column_1 ASC;
```

Notice the ORDER BY is at the end of the query. Since we usually want to do any selection or filtering before finally sorting, the ORDER BY is always going to be towards the bottom of a query and one of the last things the SQL performs.

- Use **ASC** to sort in ascending order.
- Use **DESC** to sort in descending order.
- If you leave it blank, **ORDER BY** uses **ASC** by default.

You can also ORDER BY multiple columns  
This makes sense when one column has duplicate entries.

| Company | Name   | Sales |
|---------|--------|-------|
| Apple   | Andrew | 100   |
| Google  | David  | 500   |
| Apple   | Zach   | 300   |
| Google  | Claire | 200   |

**SELECT** company,name,sales **FROM** table  
**ORDER BY** company,sales

| Company | Name   | Sales |
|---------|--------|-------|
| Apple   | Andrew | 100   |
| Apple   | Zach   | 300   |
| Google  | Claire | 200   |
| Google  | David  | 500   |

```
SELECT company, name, sales FROM table ORDER BY company, sales;
```

The data is first sorted alphabetically by company name, and then within each company, the records are sorted by sales amount in ascending order (lowest to highest).

## LIMIT

---

The **LIMIT** keyword allows us to limit the number of rows returned for a query. It is used when we do not want to return every single row in a table, but only view the top few rows to get an idea of the table layout. It is also often used in combination with **ORDER BY**. **LIMIT** goes at the very end of a query request and is the last command to be executed.

Basic Syntax Example:

```
SELECT column_1, column_2 FROM table LIMIT 3;
```

## BETWEEN

---

The **BETWEEN** operator can be used to match a value against a range of values:

- Value **BETWEEN** low **AND** high
- Which is also equivalent to: value  $\geq$  low **AND** value  $\leq$  high

Note that both the low and high here are inclusive, either both end points are included in the range.

You can also combine **BETWEEN** with the **NOT** logical operator:

- Value **NOT BETWEEN** low **AND** high
- Which is also equivalent to: value  $<$  low **OR** value  $>$  high.

Note that both the low and high here are exclusive, either both end points are not included.

The **BETWEEN** operator can also be used with dates.

- date **BETWEEN** '2007-01-01' **AND** '2007-02-01'.

Note that you need to format dates in the **ISO 8601** standard format, which is **YYYY-MM-DD**.

When using **BETWEEN** operator with dates that also include timestamp information, pay careful attention to using **BETWEEN** versus **<=, >=** comparison operators, due to the fact that a datetime starts at 0:00.

## IN

---

In some cases you want to check for multiple possible value options, for example, if a user's name shows up **IN** a list of known names.

We can use the **IN** operator to create a condition that checks to see if a value is included in a list of multiple options. This helps avoid writing a bunch of **OR** statements.

- The general syntax is:
  - **value IN (option1,option2,...,option\_n)**

Example Query:

```
SELECT color FROM table WHERE color IN ('red', 'blue', 'green')
```

Note: You can also use **IN** operator with **NOT** operator:

```
SELECT color FROM table WHERE color NOT IN ('red', 'blue')
```

## LIKE & ILIKE

---

- We've already been able to perform direct comparisons against strings, such as:
  - WHERE first\_name= 'John'
- But what if we want to match against a general pattern in a string?
  - All emails ending in '@gmail.com'
  - All names that begin with an 'A'

The **LIKE** operator allows us to perform pattern matching against string data with the use of **wildcard** characters:

- **Percent (%)** : Matches any sequence of characters.
- **Underscore (\_)**: Matches any single character.

Example usage:

- All names that begin with an 'A'
  - WHERE name LIKE 'A%'
- All names that end with an 'a'
  - WHERE name LIKE '%a'

Notice that LIKE is case-sensitive, we can use **ILIKE** which is case-insensitive

- Using the underscore allows us to replace just a single character
  - Get all Mission Impossible films
  - WHERE title LIKE 'Mission Impossible \_'

- You can use multiple underscores
- Imagine we had version string codes in the format 'Version#A4' , 'Version#B7', etc...
  - **WHERE** value **LIKE** 'Version#\_\_'

- We can also combine pattern matching operators to create more complex patterns
  - **WHERE** name **LIKE** '\_her%'
    - Cheryl
    - Theresa
    - Sherri

Note: For this course we are going to focus only of LIKE and ILIKE, but keep in mind that PostgreSQL does support full regex capabilities:

[PostgreSQL: Documentation: 18: 9.7. Pattern Matching](#)